SELENIUM PYTHON AUTOMATION

MODULE 1 — LAB WORKBOOK

Core Fundamentals, Locators, Synchronization and Advanced Web Interactions

Module	Module 1
Experiments	Assignments 1–6
Technology	Selenium WebDriver with Python
Browser	Microsoft Edge
Evidence	Labelled execution screenshots

Prepared as the module-wise laboratory record for completed Selenium Python Automation experiments.

1. Module Overview

This module develops practical Selenium WebDriver skills through six progressively structured assignments. The work begins with element identification and basic browser interaction, then introduces explicit synchronization and dynamic controls, followed by JavaScript dialogs, HTML table traversal, and browser context management using windows, tabs and iframes.

The curriculum identifies the first three assignments as core fundamentals and the next three as advanced user interactions. The completed evidence in this report is based on the labelled outputs generated for the six assignments.

Tools, Software and Concepts

• Python

• Selenium WebDriver

• Microsoft Edge

• ID, NAME, XPATH and CSS locator strategies

• WebDriverWait and Expected Conditions

• Checkboxes, dropdowns and autocomplete controls

• JavaScript Alert, Confirm and Prompt

• HTML WebTable traversal

• Iframes and multiple browser windows/tabs

• Assertions and screenshots

Learning Progression

Assignment	Primary skill	Practical outcome
1	Multiple locators	Locate fields with ID/NAME/XPATH and validate login
2	Explicit synchronization	Wait for dynamic content without time.sleep()
3	Dynamic controls	Interact with checkboxes, dropdowns and autocomplete
4	JavaScript dialogs	Accept, dismiss and enter data in browser dialogs
5	Web tables	Traverse rows and extract a matching record
6	Browser contexts	Work with iframe and new-tab contexts

2.1 Experiment 1: The Multi-Locator Challenge

Focus: Element identification and login validation

Problem Statement

Navigate to a login page and interact with the username field using By.ID, the password field using By.NAME, and the login button using By.XPATH. Validate successful login by asserting that the resulting URL contains /inventory.html.

Objective

Implement the assigned Selenium workflow and demonstrate successful browser interaction using Python.

Tools and Concepts Used

Python, Selenium WebDriver, Microsoft Edge, WebElement interaction, locators, assertions and screenshots.

Implementation / Key Code

```
driver.find_element(By.ID, "user-name").send_keys("standard_user")
driver.find_element(By.NAME, "password").send_keys("secret_sauce")
driver.find_element(By.XPATH, "//input[@id='login-button']").click()
assert "/inventory.html" in driver.current_url
```

Execution and Observation

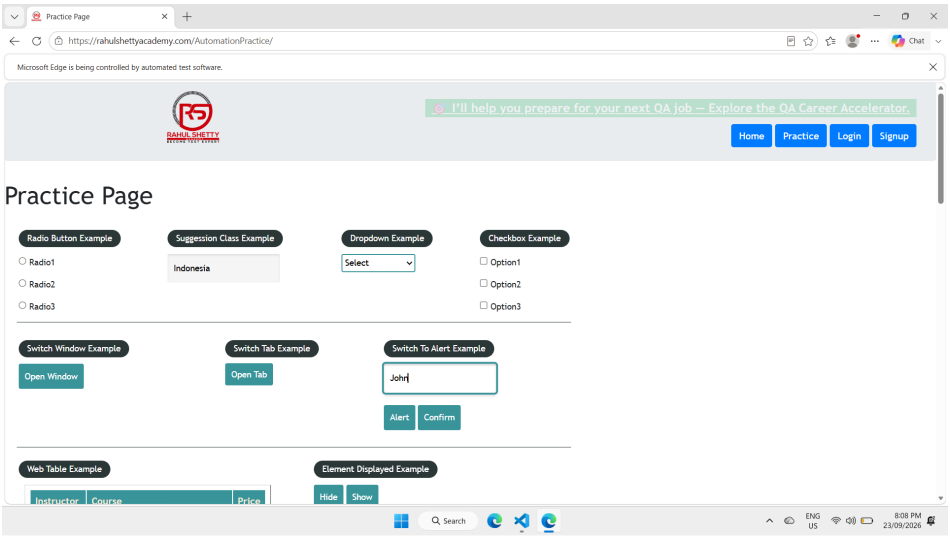
• By.ID identifies the username field.

• By.NAME identifies the password field.

• By.XPATH identifies and clicks the login button.

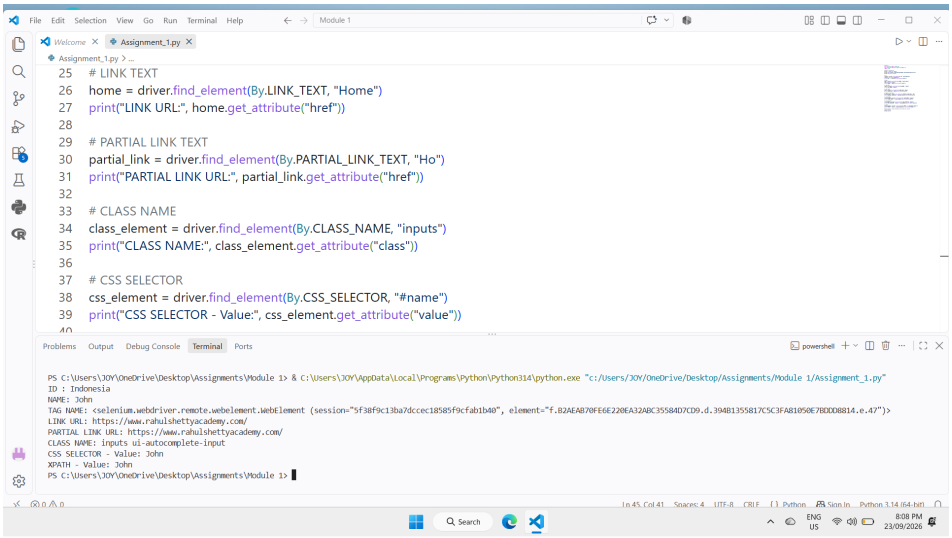
• The URL assertion verifies successful navigation.

Output Evidence



Multi Locator Practice Page

Figure 1.1 — Multi Locator Practice Page



Multi Locator Execution Output

Figure 1.2 — Multi Locator Execution Output

Result and Conclusion

Experiment 1 was completed successfully. The execution evidence demonstrates the intended Selenium interaction for **The Multi-Locator Challenge**. The exercise strengthens practical browser-automation skills and provides a foundation for the framework concepts introduced in later modules.

2.2 Experiment 2: Synchronization & Explicit Waits

Focus: Dynamic content and synchronization

Problem Statement

Navigate to a page containing dynamic content. The task requires waiting for the target text element to become visible using WebDriverWait and Expected Conditions, without using time.sleep().

Objective

Implement the assigned Selenium workflow and demonstrate successful browser interaction using Python.

Tools and Concepts Used

Python, Selenium WebDriver, Microsoft Edge, WebElement interaction, locators, assertions and screenshots.

Implementation / Key Code

```
wait = WebDriverWait(driver, 10)
element = wait.until(
    EC.visibility_of_element_located((By.ID, "finish"))
)
print(element.text)
```

Execution and Observation

• Explicit waiting synchronizes the test with the application's state.

• The wait ends when the required element becomes visible or the timeout is reached.

• This avoids fixed delays and satisfies the assignment constraint.

Output Evidence

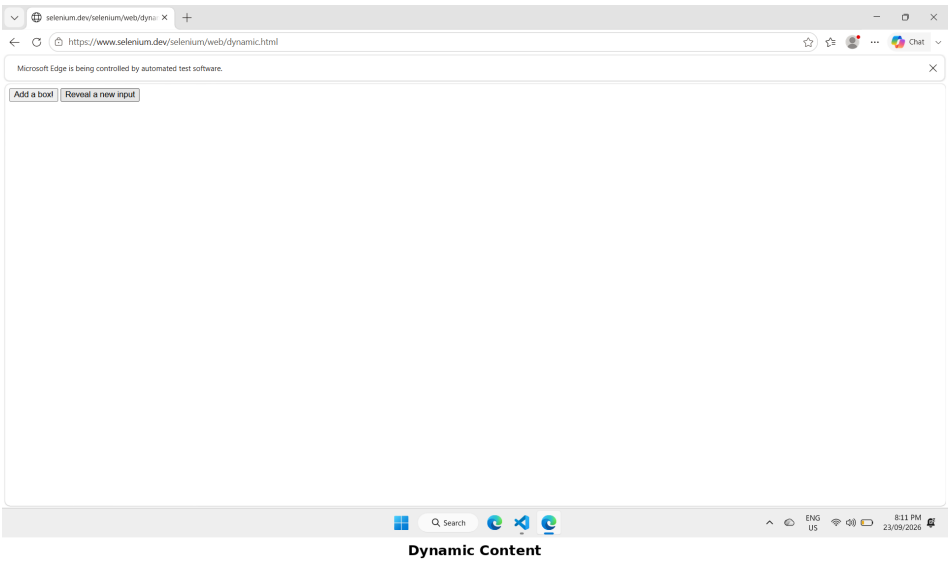


Figure 2.1 — Dynamic Content

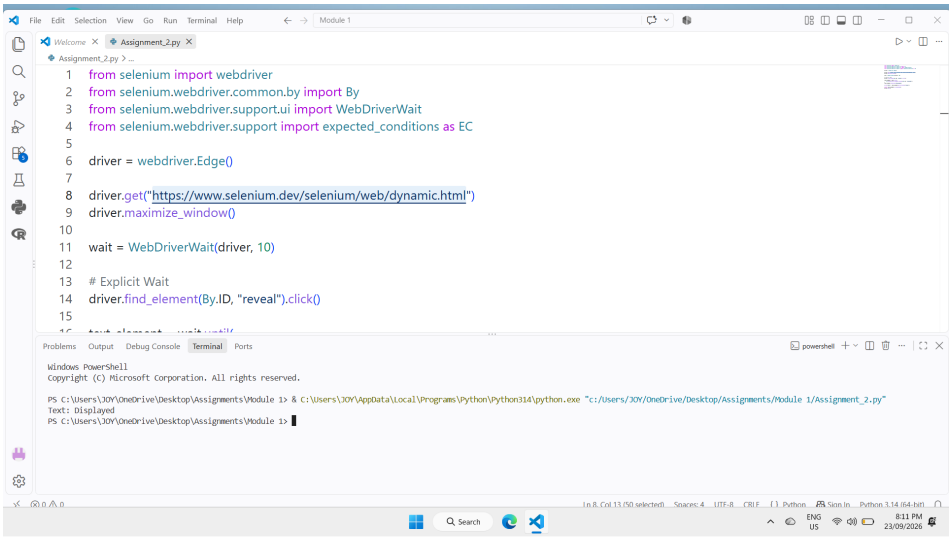


Figure 2.2 — Explicit Wait Execution Output

Result and Conclusion

Experiment 2 was completed successfully. The execution evidence demonstrates the intended Selenium interaction for **Synchronization & Explicit Waits**. The exercise strengthens practical browser-automation skills and provides a foundation for the framework concepts introduced in later modules.

2.3 Experiment 3: Dynamic Dropdowns & Checkboxes

Focus: Dynamic controls and autocomplete

Problem Statement

Interact with a form containing checkboxes, a dropdown and a search-as-you-type autocomplete list. Select required controls, verify checkbox state using is_selected(), and iterate through suggestions until the required option is found.

Objective

Implement the assigned Selenium workflow and demonstrate successful browser interaction using Python.

Tools and Concepts Used

Python, Selenium WebDriver, Microsoft Edge, WebElement interaction, locators, assertions and screenshots.

Implementation / Key Code

```
for checkbox in checkboxes:
    if not checkbox.is_selected():
        checkbox.click()

search_box.send_keys("Ind")
suggestions = driver.find_elements(By.CSS_SELECTOR, ".suggestions li")

for option in suggestions:
    if option.text == "India":
        option.click()
        break
```

Execution and Observation

• Checkbox state is verified with is_selected().

• The autocomplete list is read after entering search text.

• A loop compares suggestion text and selects the matching option.

• The workflow demonstrates interaction with controls whose available options can change dynamically.

Output Evidence

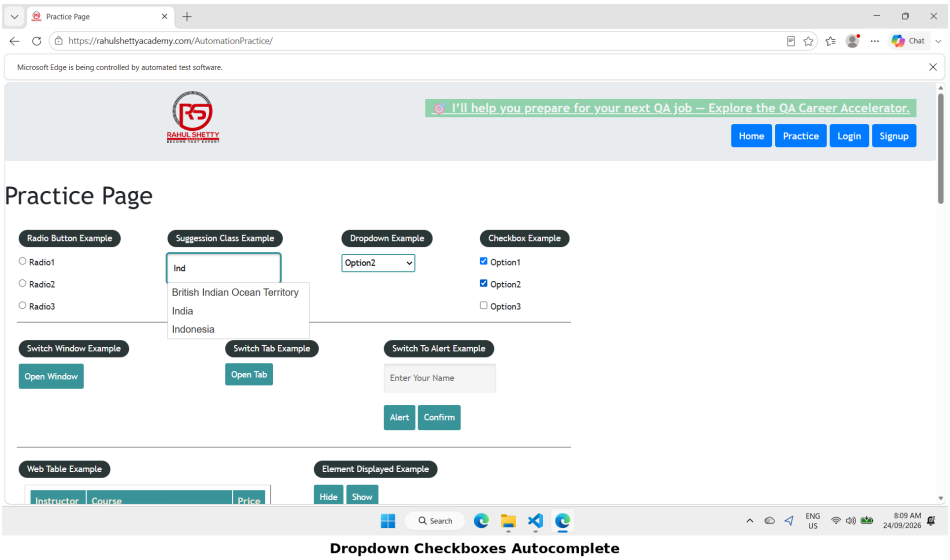


Figure 3.1 — Dropdown, Checkboxes and Autocomplete

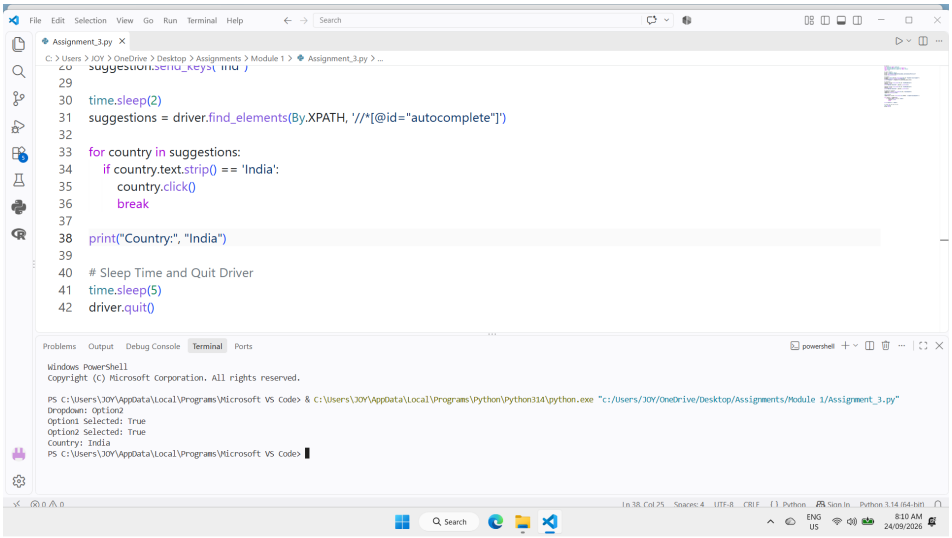


Figure 3.2 — Dynamic Controls Execution Output

Result and Conclusion

Experiment 3 was completed successfully. The execution evidence demonstrates the intended Selenium interaction for **Dynamic Dropdowns & Checkboxes**. The exercise strengthens practical browser-automation skills and provides a foundation for the framework concepts introduced in later modules.

2.4 Experiment 4: JavaScript Alerts and Confirms

Focus: Alert, Confirm and Prompt handling

Problem Statement

Trigger a JavaScript Alert, Confirm Box and Prompt Box. Accept the first alert, dismiss the confirm dialog, then enter text into the prompt and accept it.

Objective

Implement the assigned Selenium workflow and demonstrate successful browser interaction using Python.

Tools and Concepts Used

Python, Selenium WebDriver, Microsoft Edge, WebElement interaction, locators, assertions and screenshots.

Implementation / Key Code

```
alert = driver.switch_to.alert
alert.accept()

confirm = driver.switch_to.alert
confirm.dismiss()

prompt = driver.switch_to.alert
prompt.send_keys("John")
prompt.accept()
```

Execution and Observation

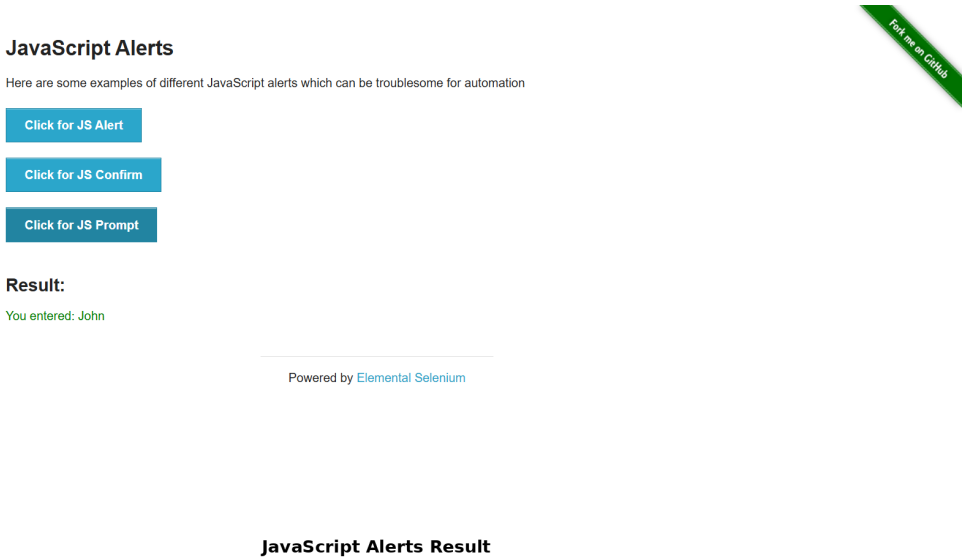
• driver.switch_to.alert changes the WebDriver context to the active dialog.

• accept() confirms an alert or prompt.

• dismiss() cancels a confirm dialog.

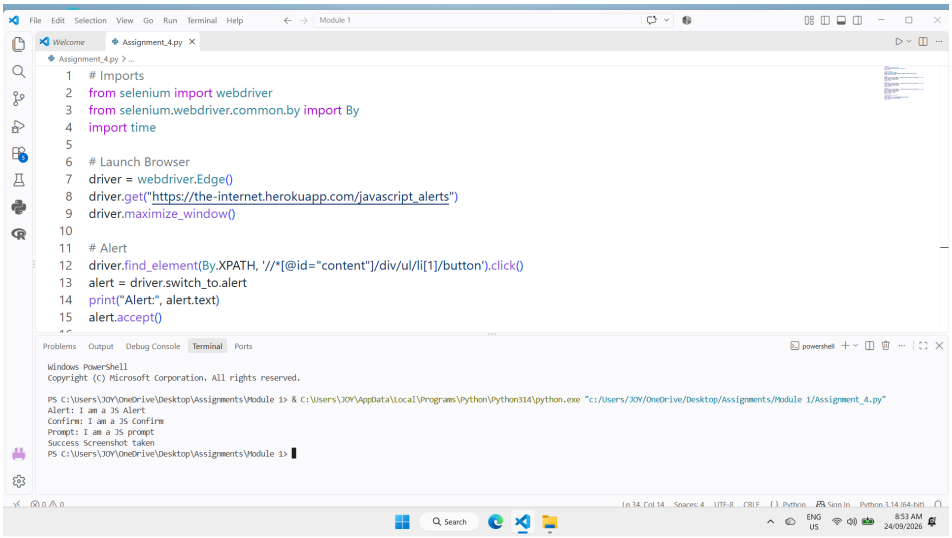
• send_keys() enters the required prompt value before submission.

Output Evidence



JavaScript Alerts Result

Figure 4.1 — JavaScript Alerts Result



JavaScript Alerts Execution Output

Figure 4.2 — JavaScript Alerts Execution Output

Result and Conclusion

Experiment 4 was completed successfully. The execution evidence demonstrates the intended Selenium interaction for **JavaScript Alerts and Confirms**. The exercise strengthens practical browser-automation skills and provides a foundation for the framework concepts introduced in later modules.

2.5 Experiment 5: HTML Web Table Extractor

Focus: Table traversal and data extraction

Problem Statement

Locate a multi-column dynamic data table, iterate through rows, identify the row matching a target name, and retrieve the corresponding value from the required column. The completed exercise used a QA Playground data table and extracted the matching record.

Objective

Implement the assigned Selenium workflow and demonstrate successful browser interaction using Python.

Tools and Concepts Used

Python, Selenium WebDriver, Microsoft Edge, WebElement interaction, locators, assertions and screenshots.

Implementation / Key Code

```
rows = driver.find_elements(By.XPATH, "//table//tbody/tr")

for row in rows:
    cells = row.find_elements(By.TAG_NAME, "td")
    if cells and cells[0].text == "Clean Code":
        print("Genre:", cells[1].text)
        break
```

Execution and Observation

• The script traverses table rows and cells rather than relying on a fixed row number.

• The target record is identified by matching its text.

• The required adjacent value is extracted from the matching row.

• The execution evidence records the extracted Genre value for Clean Code as Technology.

Output Evidence

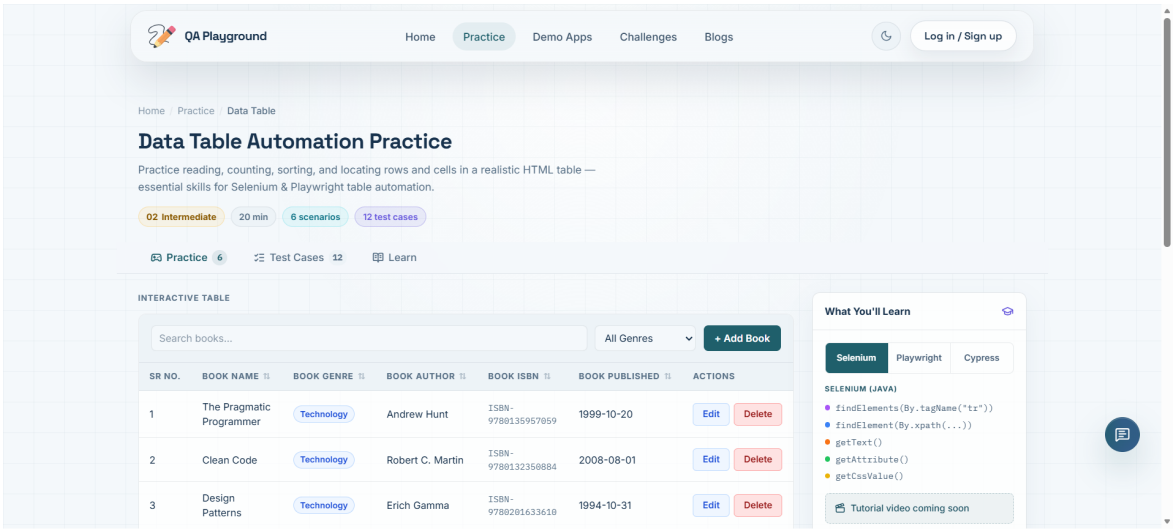
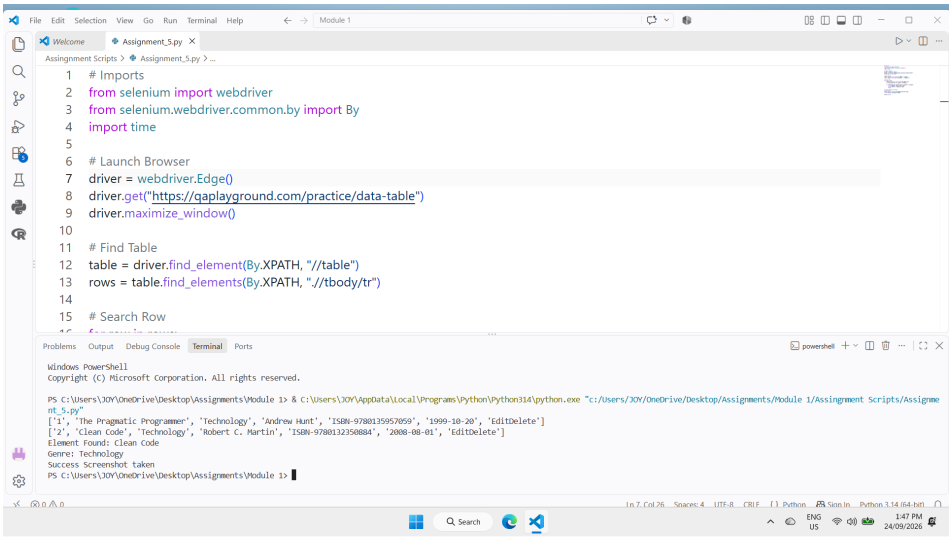


Figure 5.1 — Data Table Extraction



Data Table Execution Output

Figure 5.2 — Data Table Execution Output

Result and Conclusion

Experiment 5 was completed successfully. The execution evidence demonstrates the intended Selenium interaction for **HTML Web Table Extractor**. The exercise strengthens practical browser-automation skills and provides a foundation for the framework concepts introduced in later modules.

2.6 Experiment 6: Windows, Tabs and Iframes

Focus: Browser context management

Problem Statement

Open a page containing an embedded iframe and a button that opens a new browser tab. Switch into the iframe to interact with its content, open the new tab, use window_handles to switch context, capture the title, close the new tab and return to the main window.

Objective

Implement the assigned Selenium workflow and demonstrate successful browser interaction using Python.

Tools and Concepts Used

Python, Selenium WebDriver, Microsoft Edge, WebElement interaction, locators, assertions and screenshots.

Implementation / Key Code

```
driver.switch_to.frame(iframe)

# interact with iframe content

driver.switch_to.default_content()

main_window = driver.current_window_handle
new_tab_button.click()

for window in driver.window_handles:
    if window != main_window:
        driver.switch_to.window(window)
        break

print("New Tab Title:", driver.title)
driver.close()
driver.switch_to.window(main_window)
```

Execution and Observation

• switch_to.frame() changes context to the embedded document.

• default_content() returns to the main page.

• window_handles provides the available browser contexts.

• The new tab is identified, inspected and closed before returning to the main window.

Output Evidence

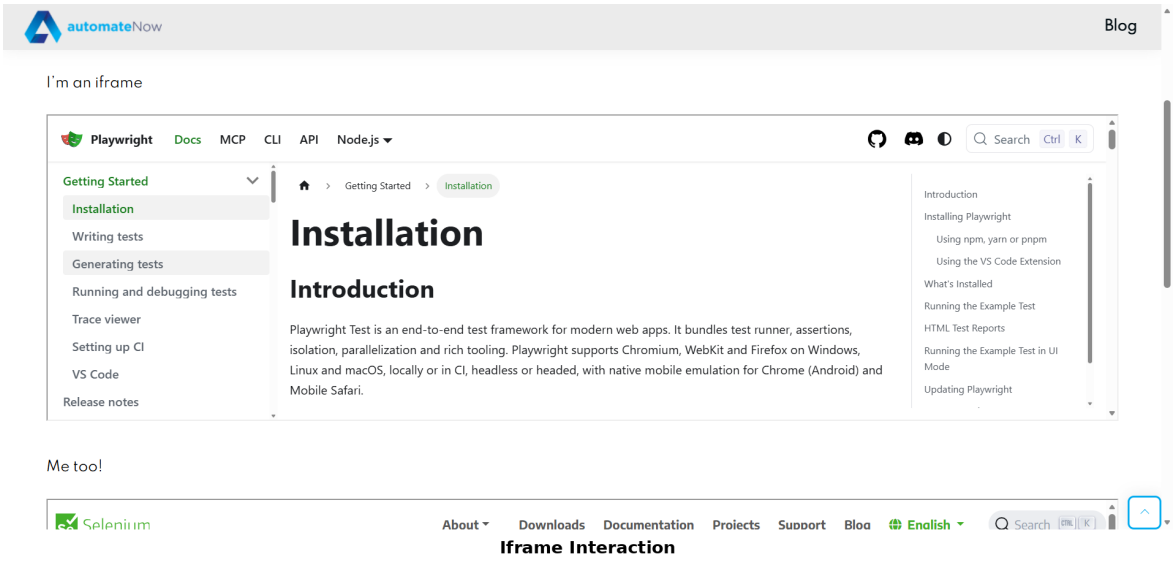


Figure 6.1 — Iframe Interaction

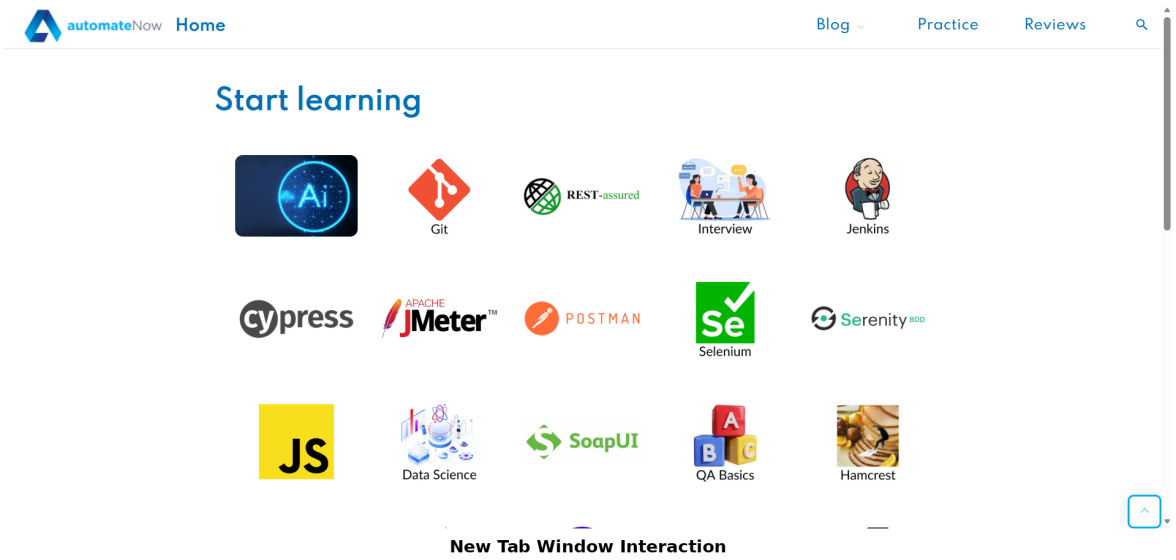


Figure 6.2 — New Tab Window Interaction

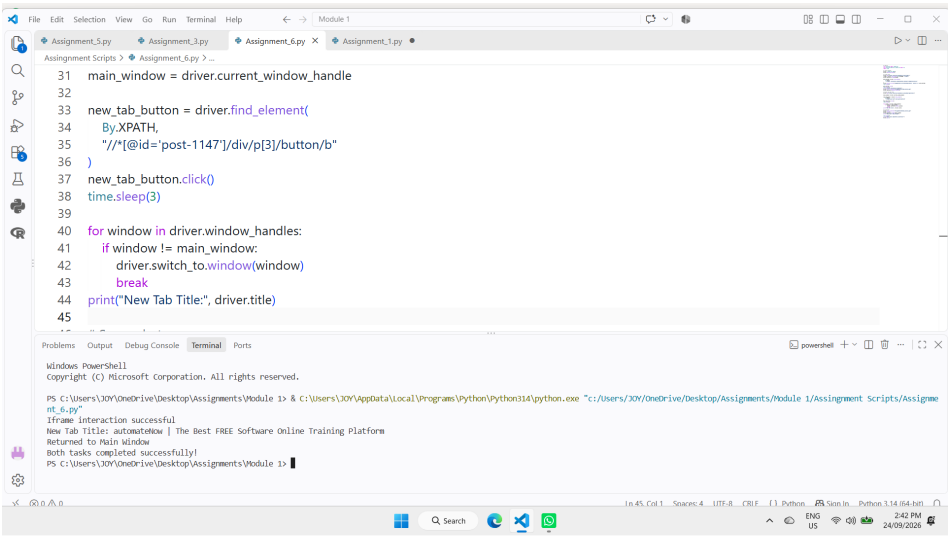


Figure 6.3 — Window and Iframe Execution Output

Result and Conclusion

Experiment 6 was completed successfully. The execution evidence demonstrates the intended Selenium interaction for **Windows, Tabs and Iframes**. The exercise strengthens practical browser-automation skills and provides a foundation for the framework concepts introduced in later modules.

8. Module-Level Outcome

Completion of Assignments 1–6 established the fundamental Selenium WebDriver workflow required for web automation. The work progressed from basic element identification to synchronization and increasingly complex browser interactions.

Key Outcomes

• Applied multiple Selenium locator strategies to identify and interact with web elements.

• Used explicit synchronization for dynamic content instead of fixed delays in the explicit-wait assignment.

• Handled checkboxes, dropdowns and autocomplete suggestions programmatically.

• Controlled JavaScript Alert, Confirm and Prompt dialogs.

• Traversed an HTML table and extracted data from a matching row.

• Switched between iframe and browser-window contexts and returned safely to the main window.

• Produced labelled screenshots as execution evidence for the completed experiments.

Overall Conclusion

Module 1 provides a practical progression from basic Selenium actions to advanced user interactions. The six completed assignments demonstrate the ability to identify web elements, synchronize automation with dynamic pages, handle common browser controls, extract structured web data and manage multiple browsing contexts. These skills form the practical foundation for the Page Object Model, data-driven testing, PyTest and Behave-based automation developed in subsequent modules.

References

• Python Automation course curriculum and lab assignment specification provided for the training programme.

• Selenium WebDriver Python API documentation for WebElement interaction, waits, alerts, frames and window handling.